

LadderLeak: взлом ECDSA при утечке менее одного бита нонса

Авторы: Diego F. Aranha (DIGIT, Орхусский университет, Дания) · Felipe Rodrigues Novaes (Университет Кампинаса, Бразилия) · Akira Takahashi (DIGIT, Орхусский университет, Дания) · Mehdi Tibouchi (NTT Corporation, Япония) · Yuval Yarom (Университет Аделаиды и Data61, Австралия)

Расширенные тезисы доклада представлены на конференции CCS 2020. Данный текст — полная версия статьи.

Аннотация

Хотя ECDSA сегодня является одной из самых популярных схем цифровой подписи, она содержит ряд подводных камней на этапе реализации — в частности, из-за крайне чувствительного характера случайного значения (называемого *нонсом*), генерируемого в алгоритме подписи. Известно, что любая малая утечка нонса или его смещение (bias) в принципе могут привести к полному восстановлению ключа: восстановление ключа при этом является частным случаем задачи о скрытом числе (hidden number problem, HNP) Боне и Венкатесана. Это наблюдение практически использовалось во множестве атак, описанных в литературе, эксплуатирующих дефекты реализации или уязвимости по сторонним каналам в конкретных реализациях ECDSA. Однако большинство атак до сих пор опирались как минимум на 2 бита смещения нонса (за исключением частного случая кривых с уровнем безопасности 80 бит, для которых известны атаки при 1-битном смещении, хотя и с очень большим числом требуемых подписей).

В настоящей работе мы раскрываем **LadderLeak** — новый класс уязвимостей по сторонним каналам в реализациях лестницы Монтгомери (Montgomery ladder), используемой при скалярном умножении в ECDSA. Уязвимость, в частности, присутствует в нескольких недавних версиях OpenSSL. При этом она приводит к утечке *менее одного бита* информации о нонсе — в том смысле, что она раскрывает старший бит нонса, но с вероятностью < 1 . Эксплуатация столь слабой утечки была бы неосуществимой с помощью методов, известных в литературе до сих пор. Однако мы представляем ряд теоретических улучшений подхода на основе анализа Фурье к решению HNP (подход, изначально предложенный Бляйхенбахером), и это позволяет нам практически взломать уязвимые к LadderLeak реализации ECDSA, инстанцированные на эллиптических кривых sect163r1 и NIST P-192. При этом мы устанавливаем несколько значимых вычислительных рекордов в практических атаках на HNP.

Ключевые слова: атака по стороннему каналу, кэш-атака, ECDSA, OpenSSL, лестница Монтгомери, задача о скрытом числе, атака Бляйхенбахера, обобщённая задача о днях рождения.

1. Введение

Алгоритм ECDSA — одна из наиболее широко применяемых схем цифровой подписи на сегодняшний день; он входит в состав многих практических криптографических протоколов, таких как TLS и SSH. Операция подписи опирается на эфемерное случайное значение, называемое *нонсом*, которое особенно чувствительно: критически важно обеспечить, чтобы нонсы сохранялись в секрете и выбирались из равномерного распределения на определённом целочисленном интервале. Легко видеть, что если нонс раскрыт или полностью повторно использован, то злоумышленник способен извлечь секретный ключ подписи, наблюдая лишь несколько подписей. Развивая это простое наблюдение, криптоаналитики обнаружили более сильные атаки, позволяющие восстановить секретный ключ, даже если утекают или смещены лишь короткие подстроки битов нонсов. Эти расширенные атаки связывают восстановление ключа с так называемой задачей о скрытом числе (HNP) Боне и Венкатесана [15] и относятся к направлению исследований, начатому Хоугрейв-Грэмом и Смартом [36], которые описали атаку на основе решёток для решения соответствующей задачи, а также Бляйхенбахером [13], предложившим подход на основе анализа Фурье.

Атаки на основе решёток, как известно, работают очень эффективно, когда злоумышленнику известны достаточно длинные подстроки нонсов (скажем, более 4 бит для подписей на 256-битной эллиптической кривой и не менее 2 бит для 160-битной кривой). В результате ряд предыдущих работ адаптировал метод Хоугрейв-Грэма и Смарт для практического взлома уязвимых реализаций ECDSA и родственных схем, таких как подписи Шнорра [60], например путём комбинирования его с анализом сторонних каналов на нонсах (см. обзор смежных работ в разделе 2.4). Однако ограничение атак на основе решёток состоит в том, что они становятся по сути неприменимыми, когда для каждого входного образца известна лишь очень малая доля нонса. В частности, для однобитной утечки нонса считается, что они должны с высокой вероятностью терпеть неудачу, поскольку вектор решётки, соответствующий секрету, уже не ожидается существенно короче других векторов решётки [6, 49]. Кроме того, подходы на основе решёток предполагают, что входные данные абсолютно точны, и очень плохо ведут себя при наличии ошибочной информации об утечке.

В отличие от этого, атака Бляйхенбахера на основе анализа Фурье способна в принципе справляться с произвольно малыми смещениями нонса и обрабатывает ошибочные входные данные, что называется, «из коробки». Несмотря на эти достоинства, она привлекла значительно меньше внимания сообщества, чем подходы на основе решёток, — отчасти, возможно, из-за отсутствия формальных публикаций, описывающих атаку, вплоть до недавнего времени (хотя некоторые рекорды атак объявлялись публично [14]). Лишь в 2013 году Де Мюлдер и соавт. [23] пересмотрели теорию подхода Бляйхенбахера в формально опубликованной научной работе, вскоре за которой последовала работа Араньи и соавт. [6], преодолевших 1-битный «решёточный барьер», взломав 160-битную ECDSA при смещении нонса всего в один бит. Такахаши,

Таблица 1. Сравнение с предыдущими рекордами решений задачи о скрытом числе при малых утечках нонса. Каждая строка соответствует размеру порядка группы, в которой инстанцирована задача. Каждый столбец соответствует максимальному

числу утѣкших битов нонса на одну подпись. Ссылки, отмеченные зелёным (соотв. фиолетовым), используют подход Бляйхенбахера (соотв. атаки на основе решѣток).

Размер	< 1	1	2	3	4
256-бит	-	-	[64]	[64]	[45, 57, 58, 70]
192-бит	эта работа	эта работа	-	-	-
160-бит	эта работа	[6, 14], эта работа (меньше данных)	[13], [41]	[47]	-

Тибуши и Абе [64] улучшили пространственную сложность атаки Бляйхенбахера и взломали qDSA [55] (вариант схемы Шнорра) при 2-битных утечках нонса. Тем не менее практичность этих работ может показаться ограниченной: действительно, Де Мюлдер и соавт. атаквали параметры (а именно 384 бита с 5-битным смещением), которые эффективнее решаются атаками на решѣтках; Аранья и соавт. требовали более 8 миллиардов подписей на входе; а Такахаси и соавт. смогли осуществить атаку лишь путѐм искусственного внесения физических сбоев в модифицированную, нестандартную реализацию.

В данной работе мы представляем первые реальные уязвимости ECDSA, которые не поддаются атакам на решѣтках, но становятся практически эксплуатируемыми методом анализа Фурье — благодаря новым теоретическим улучшениям, которые мы предлагаем по сравнению с существующей литературой.

1.1. Вклад работы

Новые кэш-тайминговые атаки на лестницу Монтгомери в OpenSSL. Сначала мы предлагаем **LadderLeak** — новый класс уязвимостей, скрывающихся в алгоритмах скалярного умножения, вызываемых ECDSA. Наша атака эксплуатирует малые различия во времени внутри реализаций лестницы Монтгомери [46], опирающихся на неподходящую обработку координат, и позволяет злоумышленнику узнать один бит секретного скаляра (что соответствует нонсам в ECDSA) с высокой вероятностью. Мы обнаружили уязвимости LadderLeak в нескольких версиях OpenSSL (в частности, в ветках 1.0.2 и 1.1.0), а также в версии 0.4.0 инструментария RELIC [5]. Мы представляем две разновидности атаки: одну для двоичных кривых и другую для простых кривых. Обе были экспериментально подтверждены и обеспечивают высокоточные различители для нонсов ECDSA в наших целевых версиях OpenSSL. В принципе уязвимость затрагивает различные параметры кривых в указанных реализациях, включая NIST P-192, P-224, P-256, P-384, P-521, B-283, K-283, K-409, B-571, sect163r1, secp192k1, secp256k1.¹ В разделе 3 мы описываем идею атаки, а также конкретные эксперименты по сторонним каналам, проведѐнные с использованием кэш-тайминговых атак Flush+Reload [73]. В качестве конкретных целей мы выбираем ECDSA, инстанцированную на NIST P-192 и sect163r1, и успешно извлекаем 1 бит информации о нонсе с высокой вероятностью.

Улучшенный теоретический анализ решения HNP по Бляйхенбахеру. В разделе 4 мы устанавливаем единую формулу компромисса «время-память-данные» для атак типа Бляйхенбахера и используем её для конкретного нахождения оптимального выбора параметров атаки при заданном размере группы и заданной величине смещения нонса. Наша формула опирается на связь между задачей о скрытом числе, с одной стороны, и *задачей о сумме K списков* (K-list sum problem) — с другой (последняя является подзадачей обобщѐнной задачи о днях рождения (GBP) Вагнера [69], хорошо известной в криптологии симметричных ключей). Наш подход универсален и позволяет в принципе интегрировать любой алгоритм суммирования K целочисленных списков для вывода аналогичной формулы компромисса. Хотя считалось, что метод Бляйхенбахера требует миллиардов подписей на входе для атаки на 1 бит утечки нонса, мы доказываем, что можно существенно снизить сложность по данным, тщательно подбирая входные параметры нашей формулы компромисса. Наш анализ также обеспечивает значительное улучшение сложности по данным для утечек более 1 бита, позволяя злоумышленнику со сторонним каналом восстановить ключ ECDSA, имея во многих случаях лишь несколько тысяч подписей, а в некоторых сценариях — даже несколько сотен. Полные оценки сложности, приведѐнные в Приложении D, поэтому могут представлять самостоятельный интерес. Мы также учитываем эффект ложного детектирования старшего бита в образцах HNP, что становится критичным при комбинировании с практической утечкой по стороннему каналу, которую мы рассматриваем.

Оптимизированная реализация и новые рекорды атак на HNP. Объединяя оба вклада, мы осуществляем полную атаку восстановления ключа подписи на подписи ECDSA, инстанцированные на двоичной кривой sect163r1 и на простой кривой NIST P-192, при утечке *менее 1 бита* нонса (в том смысле, что мы восстанавливаем 1 бит нонсов, но с вероятностью меньше 1). Разработанная нами формула компромисса позволяет взломать соответствующие экземпляры HNP при реалистичных вычислительных ресурсах. В наших экспериментах по атаке, представленных в разделе 5, сложность по данным, требуемая для первого случая, значительно меньше, чем у Араньи и соавт. [6], — примерно в 2¹⁰ раз. Более того, насколько нам известно, 192-битная ECDSA никогда прежде не взламывалась при утечке в 1 бит или менее (см. Таблицу 1 для сравнения с предыдущими рекордами HNP), и поэтому наша конкретная атака знаменует резкий прогресс в практических атаках на HNP. Это стало возможным благодаря настройке компромиссов для оптимизации временной сложности и запуску нашей высокооптимизированной масштабируемой реализации в облачных инстансах Amazon Web Service (AWS) EC2. Подход и реализация, разработанные в этой работе, применимы к различным типам утечек из ECDSA независимо от уязвимости LadderLeak и потому открывают интересное направление для будущей криптоаналитической работы. Например, наши эмпирические результаты также указывают, что взлом более крупных экземпляров вроде P-224 с 1-битной утечкой или P-256 с 2-битной утечкой был бы практически осуществим при относительно скромной сложности по данным. Экспериментальные результаты по кэш-атакам, представленные патчи с контрмерами, решатель формулы компромисса и оптимизированная реализация атаки Бляйхенбахера доступны в нашем репозитории на GitHub.²

¹ OpenSSL по умолчанию не вызывает уязвимую реализацию лестницы для P-256 и вместо этого опирается на специальный код, включаемый при компиляции. Специальный код может быть аналогично включѐн на этапе сборки для P-224 и P-521 с помощью ключа `enable-ec_nistp_64_gcc_128`. Мы благодарим анонимного рецензента за указание на эти особенности поведения OpenSSL.

² <https://github.com/akiratk0355/ladderleak-attack-ecdsa>

1.2. Уязвимые версии ПО и скоординированное раскрытие

В декабре 2019 года мы первоначально сообщили команде разработчиков OpenSSL об уязвимостях в версиях 1.0.2t и 1.0.1l в соответствии с политикой безопасности OpenSSL³ — до окончания долгосрочной поддержки этих версий. После выхода версии 1.0.2u мы подтвердили, что те же уязвимости по-прежнему присутствуют, и потому предложили патч с соответствующими контрмерами, который уже был одобрен.⁴ Хотя ветка 1.0.2 с мая 2020 года более не поддерживается публично, команда разработчиков OpenSSL по-прежнему предоставляет своим клиентам расширенную поддержку для 1.0.2, и наш патч включён в их дальнейшие внутренние выпуски. Хотя трудно оценить, как уязвимость затронула бы реальные продукты из-за отсутствия публичных выпусков с патчем, поиск по GitHub выявил несколько проектов, обновляющих процесс сборки для учёта нового выпуска из ветки 1.0.2. Аналогичные шаги были предприняты и для устранения уязвимости в RELIC, и исправление было внесено в январе 2020 года.

³ <https://www.openssl.org/policies/secpolicy.html> ⁴ <https://github.com/openssl/openssl/pull/11361>

2. Предварительные сведения

Обозначения. Мнимую единицу обозначаем через i . Для любого положительного целого x функция $\text{MSB}_n(x)$ возвращает его n старших битов. Когда a и b — целые числа такие, что $a < b$, мы используем обозначение целочисленного интервала $[a, b]$ для множества $\{a, a + 1, \dots, b\}$. Всюду в статье $\log$ с обозначает двоичный логарифм $c > 0$.

2.1. Кэш-атаки

Чтобы обеспечить высокую производительность, ожидаемую от современных компьютеров, процессоры применяют массу приёмов, направленных на предсказание поведения программы и оптимизацию процессора под это поведение. Как прямое следствие, выполнение программы влияет на внутреннее состояние процессора, что, в свою очередь, влияет на скорость будущего выполнения программ. Таким образом, отслеживая собственную производительность, программа может узнавать о паттернах выполнения других программ, создавая непреднамеренный и неконтролируемый канал связи [29].

Непреднамеренная утечка паттернов выполнения криптографического ПО может иметь разрушительное воздействие на безопасность реализации. За годы множество атак продемонстрировали полное восстановление ключа за счёт эксплуатации утечек через внутреннее состояние различных микроархитектурных компонентов, включая кэши [1, 40, 51, 66], предсказатели ветвлений [2, 39], функциональные блоки [3, 18] и таблицы трансляции [31].

Flush+Reload [34, 73] — заметная стратегия атаки, в которой злоумышленник отслеживает доступ жертвы к некоторой ячейке памяти. Атака состоит из двух шагов. На шаге *flush* злоумышленник вытесняет отслеживаемую ячейку памяти из памяти, обычно с помощью специальной инструкции вроде `clflush`. Затем злоумышленник некоторое время выжидает, чтобы дать жертве время выполниться. Наконец, на шаге *reload* злоумышленник обращается к ячейке памяти, измеряя, как долго длится доступ. Если жертва не обращалась к отслеживаемой ячейке, та останется некешированной, и доступ будет медленным. Наоборот, если жертва обратилась к отслеживаемой ячейке между шагами *flush* и *reload*, ячейка памяти окажется закешированной, и доступ будет быстрым. Повторяя атаку, злоумышленник может восстановить паттерны использования памяти жертвой, что позволяет проводить атаки на симметричные шифры [34, 37], на примитивы с открытым ключом — как традиционные [10, 52, 73], так и постквантовые [32, 53], а также на некриптографическое ПО [33, 71].

2.2. Лестница Монтгомери и её безопасная реализация

Эллиптическая кривая E , определённая над конечным полем F , — это множество решений $(x, y) \in F$, удовлетворяющих уравнению кривой, вместе с бесконечно удалённой точкой O . Правило хорды и касательной задаёт групповой закон ($\oplus$) для сложения и удвоения точек на кривой, где O — единичный элемент. Для $P \in E(F)$ и $k \in \mathbb{Z}$ операция скалярного умножения вычисляет $R = [k]P$, что соответствует сложению P с самим собой $(k - 1)$ раз. Криптографические протоколы опираются на умножение на секретный скаляр как на фундаментальную операцию, чтобы основывать безопасность на задаче дискретного логарифмирования на эллиптической кривой (ECDLP): найти k по входным данным $(P, [k]P)$. Конкретные экземпляры эллиптических кривых, используемых в криптографии, применяют подгруппу точек большого простого порядка q , для которого ECDLP считается труднорешаемой. Из соображений эффективности принято представлять точки в проективных координатах (X, Y, Z) и избегать вычисления дорогостоящих инверсий поля при вычислении группового закона.

Во многих постановках злоумышленник, способный получить утечку из операции скалярного умножения — в частности, битов скаляра, — может существенно снизить усилия, необходимые для решения ECDLP. Лестница Монтгомери, изначально предложенная для ускорения метода факторизации ECM [46], позднее стала важной для безопасных реализаций криптографии на эллиптических кривых благодаря присущей ей регулярности в способе обработки скаляра: требуется одно и то же число групповых операций независимо от битового паттерна k . Алгоритм 1 иллюстрирует эту идею. Существует богатая литература о системах координат и моделях эллиптических кривых для безопасной реализации алгоритма [21, 50]. К сожалению, эти методы не всегда работают для стандартизованных кривых в модели Вейерштрасса, которые в значительной мере способствовали внедрению эллиптических кривых в промышленность через стандарты SECG и NIST [38].

Реализация лестницы Монтгомери с постоянным временем, защищённая от тайминговых атак, должна удовлетворять трём базовым предусловиям: (i) число итераций цикла должно быть фиксированным; (ii) операции с памятью не могут зависеть от битов секретного скаляра, чтобы избежать утечки через иерархию памяти; (iii) групповой закон должен вычисляться с одинаковым числом и типом операций поля в одном и том же порядке независимо от битов скаляра. Первое проще обеспечить, условно прибавляя q или $2q$ до тех пор, пока результирующий скаляр $\hat{k}$ не получит фиксированную длину. Отметим, что этот подход имеет интересный побочный эффект: он сохраняет старший бит k во втором старшем бите $\hat{k}$, когда q лишь немного меньше степени двойки. Второе может быть достигнуто простой заменой ветвлений на условные операции для перестановки

аккумуляторов (R_0, R_1). Третье более сложно, но значительно упрощается *полными законами сложения*, которые вычисляют корректный результат для всех возможных входных данных [12] (даже когда точка удваивается) без каких-либо исключений или особых случаев. Хотя полные законы сложения для кривых Вейерштрасса существуют [54], они влекут существенные потери производительности и недостаточно популяризированы для текущих реализаций классических стандартизованных кривых. Версия алгоритма с применёнными контрмерами приведена в Приложении А. Как мы далее обсуждаем в разделе 3, попытки улучшить защищённость от сторонних каналов в текущих реализациях скалярного умножения рискуют «надстроить» в остальном постоянно-временную лестницу Монтгомери поверх реализации группового закона, которая всё ещё допускает утечку информации через оптимизации.

Алгоритм 1. Лестница Монтгомери

```
Вход:  $k \in \mathbb{Z}_q$ , точка  $P$  на  $E(F)$ .
Выход: результат скалярного умножения  $R = [k]P$ .
1:  $(R_0, R_1) \leftarrow (P, 2P)$ 
2: для  $i = \lfloor \lg(k) \rfloor - 1$  до 0:
3:   если  $k_i = 0$ :
4:      $(R_0, R_1) \leftarrow ([2]R_0, R_0 \oplus R_1)$ 
5:   иначе:
6:      $(R_0, R_1) \leftarrow (R_0 \oplus R_1, [2]R_1)$ 
7:   конец если
8: конец цикла
9: вернуть  $R_0$ 
```

2.3. ECDSA и задача о скрытом числе с ошибочным входом

Извлечение ключа подписи из утечек нонса в подписях ECDSA обычно сводится к решению так называемой задачи о скрытом числе (HNP). Мы представляем обобщённый вариант исходной HNP Боне и Венкатесана [15], включающий некоторую ошибочную информацию о старших битах. В частности, приведённое ниже распределение ошибок моделирует ложное детектирование злоумышленником при сборе данных по стороннему каналу.

Определение 2.1 (Задача о скрытом числе с ошибочным входом). Пусть q — простое число и $sk \in \mathbb{Z}_q$ — секрет. Пусть h_i и k_i — равномерно случайные элементы в $\mathbb{Z}_q$ для каждого $i = 1, \dots, M$, и определим $z_i = k_i - h_i \cdot sk \bmod q$. Предположим некоторое фиксированное распределение χ_b на $\{0, 1\}^b$ для $b > 0$ и определим вероятностный алгоритм $\text{EMSB}_{\chi_b}(x)$, который возвращает $\text{MSB}_b(x) \oplus e$ для некоторой строки битов ошибки e , выбранной из χ_b . По данным (h_i, z_i) и $\text{EMSB}_{\chi_b}(k_i)$ для $i = 1, \dots, M$ HNP с распределением ошибок χ_b требует найти sk .

В наших конкретных атаках на ECDSA в OpenSSL мы сосредотачиваемся на случае, когда $b = 1$ и χ_b — распределение Бернулли B_ϵ для некоторого параметра частоты ошибок $\epsilon \in [0, 1/2]$, то есть $\text{EMSB}_{\chi_b}(x)$ просто возвращает отрицание старшего бита x с вероятностью ϵ , а иначе возвращает корректный бит.

Прямое вычисление показывает, что набор подписей ECDSA с утекающими нонсами действительно является экземпляром HNP. Заметим, что подпись ECDSA (r, s) , сгенерированная как в Алгоритме 2, удовлетворяет $s = (H(\text{msg}) + r \cdot sk)/k \bmod q$ для равномерно выбранного $k \in \mathbb{Z}_q$. Перегруппировывая члены, получаем:

$$s \cdot H(\text{msg}) = k - (r/s) \cdot sk \bmod q.$$

Следовательно, полагая $z = H(\text{msg})/s \bmod q$ и $h = r/s \bmod q$, мы получаем образец HNP, если старший бит k утекает с некоторой вероятностью.

Алгоритм 2. Генерация подписи ECDSA

```
Вход: ключ подписи  $sk \in \mathbb{Z}_q$ , сообщение  $\text{msg} \in \{0,1\}^*$ , порядок группы  $q$ , базовая точка  $G$ , криптографическая хеш-функция  $H: \{0,1\}^* \rightarrow \mathbb{Z}_q$ .
Выход: корректная подпись  $(r, s)$ 
1:  $k \leftarrow \mathbb{Z}_q$ 
2:  $R = (r_x, r_y) \leftarrow [k]G$ ;  $r \leftarrow r_x \bmod q$ 
3:  $s \leftarrow (H(\text{msg}) + r \cdot sk)/k \bmod q$ 
4: вернуть  $(r, s)$ 
```

2.4. Атаки на HNP на основе решёток

Боне и Венкатесан [15] предлагают решать задачу о скрытом числе, сначала сводя её к решёточной задаче о ближайшем векторе (CVP). Хоугрейв-Грэм и Сمارт [36] показывают сведение частичной утечки нонса из DSA к HNP, которую они решают сведением к CVP. Нгуен и Шпарлинский [47] доказывают, что подход Хоугрейв-Грэма и Смарт работает при утечке $\log \log q$ битов из полиномиального числа эфемерных ключей. Позднее они распространяют результат на ECDSA [48].

Брамли и Тувери [17] демонстрируют тайминговую атаку на OpenSSH, получая небольшое число битов из эфемерных ключей. Последующие работы представляют атаки через электромагнитное излучение [8, 30], кэш [4, 9, 52, 68] и другие микроархитектурные каналы [18], — все они используют преобразование HNP в решёточную задачу. Далл и соавт. [22] исследуют осуществимость решения HNP с ошибками с помощью решёточной атаки. Одна из главных целевых кривых в нашей работе — NIST P-192, которая также эксплуатировалась Медведом и Освальдом [43] с использованием нескольких битов нонса, полученных через шаблонные SPA-атаки.

2.5. Каркас атаки Бляйхенбахера

Подход к HNP на основе анализа Фурье был впервые предложен Бляйхенбахером [13] и использовался для взлома ECDSA, схемы Шнорра и их вариантов при малых утечках нонса, которые трудно эксплуатировать методом на основе решёток [6, 24, 64]. Этот раздел охватывает основы каркаса Бляйхенбахера, резюмированного в Алгоритме 3. За более подробными теоретическими деталями отсылаем к вышеупомянутым работам. Суть метода — количественно оценить *модулярное смещение* нонса k с

помощью функций смещения в форме обратного дискретного преобразования Фурье (iDFT).

Определение 2.2 (Функции смещения). Пусть K — случайная величина над $\mathbb{Z}_q$. Модулярное смещение $B_q(K)$ определяется как

$$B_q(K) = \mathbb{E}[e^{(2\pi i K/q)}]$$

где $\mathbb{E}(K)$ обозначает математическое ожидание, а i — мнимая единица. Аналогично, выборочное смещение множества точек $K = \{k_i\}_{i=1}^M$ в $\mathbb{Z}_q$ определяется как

$$B_q(K) = \frac{1}{M} \sum_{i=1}^M e^{(2\pi i k_i/q)}$$

Когда l старших битов K зафиксированы на некоторой константе, а K в остальном равномерна по модулю q , известно, что норма смещения $|B_q(K)|$ сходится к $2^l \cdot \sin(\pi/2^l)/\pi$ для больших q [64, Следствие 1]. Оценка справедлива и для выборочного смещения $B_q(K)$ для заданного набора смещённых нонсов $\{k_i\}$. Например, если первый старший бит каждого k_i зафиксирован на константе, то смещение оценивается как $|B_q(K)| \approx 2/\pi \approx 0,637$. Более того, если k_i следуют равномерному распределению над $\mathbb{Z}_q$, то среднее нормы выборочного смещения оценивается как $1/\sqrt{M}$, что является прямым следствием хорошо известного факта о среднем расстоянии от начала координат при случайном блуждании на комплексной плоскости [6].

Малые линейные комбинации. Имея такую функцию, было бы естественно предложить наивный подход к нахождению sk: для каждого кандидата секретного ключа $w \in \mathbb{Z}_q$ вычислить соответствующий набор кандидатов нонсов $K_w = \{z_i + h_i w \bmod q\}$ и затем заключить, что $w = \text{sk}$, если выборочное смещение $|B_q(K_w)|$ показывает пиковое значение. Это, конечно, не лучше исчерпывающего перебора по всей $\mathbb{Z}_q$. Чтобы избежать этой проблемы, требуется так называемый *поиск коллизий* входных образцов — критический предварительный шаг для расширения ширины пика. Де Мюлдер и соавт. [24] и Аранья и соавт. [6] показали, что ширина пика расширяется приблизительно до q/L_{FFT} за счёт взятия линейных комбинаций входных образцов $\{(h_i, z_i)\}$ для генерации новых образцов $\{(h'_j, z'_j)\}$ таких, что $h'_j < L_{\text{FFT}}$. Таким образом можно попасть куда-то в расширенный пик, проверяя выборочные смещения лишь в L_{FFT} точках-кандидатах над $\mathbb{Z}_q$. Поскольку обратное DFT в L_{FFT} точках может быть эффективно вычислено алгоритмами быстрого преобразования Фурье (FFT) за время $O(L_{\text{FFT}} \log L_{\text{FFT}})$ и с памятью $O(L_{\text{FFT}})$, первая цель фазы поиска коллизий — найти достаточно малые линейные комбинации образцов, чтобы FFT на таблице размера L_{FFT} стало практически вычислимым. Для фазы поиска коллизий исследовались разные подходы, такие как редукция решёток [24], сортировка-и-разность (sort-and-difference) [6] и алгоритм суммы 4 списков [64].

Разреженные линейные комбинации. Может возникнуть искушение повторять такие операции поиска коллизий столько раз, сколько нужно, пока не будут получены линейные комбинации с желаемой верхней границей. Однако линейные комбинации имеют свою цену: в обмен на бóльшую ширину пика его высота экспоненциально уменьшается. Конкретно, если исходное модулярное смещение нонса равно $|B_q(K)|$ и все коэффициенты в линейных комбинациях ограничены множеством $\{-1, 0, 1\}$, то смещение пика возводится в степень, равную L_1 -норме вектора коэффициентов [64]. Таким образом, чтобы высота пика была отличима от шумового значения, уменьшённый пик должен быть значительно больше шумового значения (которое в среднем равно $1/\sqrt{M}$, как упомянуто выше). Это накладывает ещё одно ограничение на фазу поиска коллизий — *разреженность* линейных комбинаций. Итого, эффективность атаки Бляйхенбахера критически зависит от сложностей алгоритма поиска малых и разреженных линейных комбинаций.

2.6. Задача о сумме K списков

Мы вводим вариант *задачи о сумме K списков* [25] (подзадачи GBP [69]), инстанцированный над целыми числами. Дальнейшая часть статьи обсуждает связь между этой задачей и каркасом Бляйхенбахера.

Определение 2.3 (Задача о сумме K списков). Даны K отсортированных списков $L_1, \dots, L_K$, каждый из которых состоит из 2^a равномерно случайных ℓ -битных целых чисел; задача о сумме K списков требует найти непустой список L' , состоящий из $x' = \sum_{i=1}^K \omega_i x_i$, где K -кортежи $(x_1, \dots, x_K) \in L_1 \times \dots \times L_K$ и $(\omega_1, \dots, \omega_K) \in \{-1, 0, 1\}^K$ удовлетворяют $\text{MSB}_n(x') = 0$ для некоторого целевого параметра $n \leq \ell$.

Алгоритм 4 — это экземпляр алгоритма суммы K списков для $K = 4$. Это по сути параметризованный вариант алгоритма Хоугрейв-Грэма-Жу [35], который мы анализируем, расширяя каркас Динура [25] в разделе 4.

Алгоритм 3. Каркас атаки Бляйхенбахера

```
Вход:
  {(h_i, z_i)} — M образцов HNP над Z_q.
  M' — число линейных комбинаций, которые нужно найти.
  L_FFT — размер таблицы FFT.
Выход: старшие биты sk
1: Поиск коллизий
2: Сгенерировать M' образцов {(h'_j, z'_j)}, где
   (h'_j, z'_j) = (\sum_i \omega_{i,j} h_i, \sum_i \omega_{i,j} z_i) — пара линейных комбинаций
   с коэффициентами \omega_{i,j} \in \{-1, 0, 1\}, такими что для j \in [1, M']:
   (1) Малость: 0 \le h'_j < L_FFT и
   (2) Разреженность: |B_q(K)|^{|\Omega_j|} \gg 1/\sqrt{M'} для всех j,
       где \Omega_j := \sum_i |\omega_{i,j}|.
3: Вычисление смещения
4: Z := (Z_0, ..., Z_{L_FFT-1}) \leftarrow (0, ..., 0)
5: для j = 1 до M':
6:   Z_{h'_j} \leftarrow Z_{h'_j} + e^{(2\pi i z'_j / q)i}
7: конец цикла
8: {B_q(K_{w_i})} \leftarrow FFT(Z), где w_i = i \cdot q / L_FFT.
9: Найти значение i, при котором |B_q(K_{w_i})| максимально.
10: Вывести старшие log L_FFT битов w_i.
```

Алгоритм 4. Параметризованный алгоритм суммы 4 списков на основе Хоугрейв-Грэма-Жу [35]

```
Вход:
{Li} (i=1..4) — отсортированные списки из 2a равномерно случайных  $\ell$ -битных образцов.
n — число обнуляемых старших битов в каждом раунде.
v ∈ [0, a] — параметр.
Выход: L' — список (ℓ - n)-битных образцов.
1. Для каждого c ∈ [0, 2v):
  а. Найти пары (x1, x2) ∈ L1 × L2 такие, что MSBa(x1 + x2) = c.
  Сохранить ожидаемое число 2{2a-a} = 2a выходных сумм x1 + x2
  в новый отсортированный список L'1. То же для L3 и L4 → L'2.
  б. Найти пары (x'1, x'2) ∈ L'1 × L'2 такие, что MSBn(|x'1 - x'2|) = 0.
  Сохранить ожидаемое число 2{2a-(n-a)} = 2{3a-n}
  выходных сумм |x'1 - x'2| в список L'.
2. Вывести L' ожидаемой длины M' = 2{3a+v-n}.
```

3. Тайминговые атаки на лестницу Монтгомери

Реализация лестницы Монтгомери должна быть построена поверх постоянно-временной реализации группового закона, чтобы пользоваться её гарантиями устойчивости к сторонним каналам. Любое малейшее отклонение в числе операций поля или в паттерне доступа к памяти в групповом законе может привести к утечке информации о том, какая из двух ветвей конкретной итерации выполняется, что далее раскрывает информацию о бите ключа. В этой работе мы эксплуатируем уязвимость в способе подготовки лестницы Монтгомери (строка 1 Алгоритма 1), замечая, что реализации, использующие проективные координаты, будут иметь аккумулятор R_0 в аффинных координатах, в которых обычно задаётся входная точка P , и R_1 в проективных координатах после выполненного удвоения точки. Это несоответствие координат позволяет злоумышленнику осуществить кэш-тайминговую атаку на первую итерацию лестницы, раскрывая второй старший бит скаляра. Мы обнаружили эту проблему в популярной криптографической библиотеке OpenSSL и в исследовательски-ориентированном инструментарии RELIC [5], — обе, по-видимому, вызваны попыткой реализовать постоянно-временную лестницу поверх группового закона, оптимизированного для особых случаев. Эта общность побудила нас назвать обнаруженную уязвимость условным именем **LadderLeak**.

3.1. Кэш-тайминговые уязвимости в реализации OpenSSL

OpenSSL содержит в своей кодовой базе несколько реализаций лестницы Монтгомери в зависимости от выбора кривой и поля, поэтому мы разделим обсуждение по выбору лежащего в основе поля.

Двоичные кривые. Для кривых, определённых над F_{2^m} , OpenSSL применяет хорошо известный алгоритм скалярного умножения Лопеса-Дахаба [42], что сводится к лестнице Монтгомери над точками, представленными в координатах Лопеса-Дахаба ($x = X/Z$, $y = Y/Z^2$). Затронутые параметры: кривая SECG sect163r1; и кривые NIST B-283, K-283, K-409 и B-571 (то есть двоичные кривые с порядком группы чуть ниже степени двойки) в версиях 1.0.2u и 1.1.0l. Последняя ветка 1.1.1 не затронута благодаря унифицированной и защищённой реализации лестницы.

Лестница Монтгомери реализована в функции `ec_GF2m_montgomery_point_multiply()` в файле `crypto/ec/ec2_mult.c`. Функция вычисляет скалярное умножение $[k]P$ для скаляра k фиксированной длины и входной точки $P = (x, y)$. Лестница начинается с инициализации двух точек $(X_1, Z_1) = (x, 1)$ и $(X_2, Z_2) = [2]P = (x^4 + b, x^2)$. Первая итерация цикла следует за функцией условной перестановки, которая обменивает эти две точки на основе значения второго старшего бита. Первая функция, вызываемая внутри первой итерации, — `gf2m_Madd()` для сложения точек, которая начинается с умножения на значение Z_1 . Однако, поскольку арифметика конечного поля реализована не с постоянным временем для двоичных полей, существует различие во времени между умножением на (1) и на (x^2) , так как модулярная редукция нужна лишь в последнем случае. В частности, модулярная редукция будет вычисляться, когда $Z_1 = x^2$ после условной перестановки. Это происходит, когда второй старший бит равен 1, поскольку условная перестановка фактически поменяла местами два набора значений. Кэш-тайминговая атака может затем отслеживать, когда вызывается код модулярной редукции для приведения нетривиального промежуточного результата умножения.

Простые кривые. В кривых, определённых над F_p для большого простого p , OpenSSL 1.0.2u применяет лестницу Монтгомери, когда предвычисление отключено, — сценарий, распространённый на практике, поскольку предвычисление должно быть вручную включено для определённой точки (обычно фиксированного генератора) [67]. Затронутые параметры: кривые NIST P-192, P-224, P-384 и P-521; и кривые SECG secp192k1 и secp256k1 (то есть простые кривые с порядком группы чуть ниже степени двойки). Кривая P-256 затронута в принципе, но OpenSSL по умолчанию включает специальный код на этапе сборки. Отметим, что secp256k1 — это кривая, принятая для подписи биткойн-транзакций с помощью ECDSA.

В этом случае OpenSSL реализует лестницу Монтгомери, используя оптимизированные формулы для арифметики эллиптических кривых в модели Вейерштрасса. Лестница реализована в `ec_mul_consttime()` в `/crypto/ec/ec_mult.c`, которая, однако, не работает с постоянным временем с точки зрения кэша, несмотря на название. Лестница начинается с инициализации двух аккумуляторов $R_0 = P$ (в аффинных координатах) и $R_1 = 2P$ (в проективных координатах). Первая итерация цикла нетривиальна и вычисляет сложение точек и удвоение точки после условной перестановки. В зависимости от бита ключа условная перестановка эффективна, и только одна точка останется хранимой в проективных координатах. И функция сложения точек, и функция удвоения точки имеют встроенные оптимизации для смешанного сложения, и Z -координата входной точки может быть обнаружена в случае удвоения точки, реализованном в функции `ec_GFp_simple_dbl()`. Когда входная точка для функции удвоения находится в аффинных координатах, умножение на Z заменяется более быстрым вызовом `BN_copy()`. Это происходит, когда два аккумулятора не переставлены в лестнице, что означает, что точка R_0 в аффинных координатах удваивается и второй старший бит равен 0. Различие во времени очень мало, но может быть обнаружено кэш-тайминговой атакой.

3.2. Реализация атак

Мы реализовали кэш-тайминговые атаки с использованием Flush+Reload из программы `FR-trace`, доступной в инструментарии

анализа сторонних каналов Mastik [72]. Мы атаковали OpenSSL, запуская вычисление подписи из командной строки на двух процессорах Broadwell моделей Core i7-5500U и i7-3520M с тактовой частотой 2,4 ГГц и 2,9 ГГц соответственно и с отключённым TurboBoost. OpenSSL был собран со стандартной конфигурацией с включёнными символами отладки и оптимизациями. Целевые параметры были на наименьшем уровне безопасности в каждом классе: sect163r1 для двоичного случая и P-192/secp192k1 для простого. Хотя наблюдаемое различие во времени было очень малым в обоих случаях, нам удалось усилить его с помощью *деградации производительности* [4]: несколько потоков, работающих в фоне, штрафуют целевые участки кода (модулярная редукция в двоичном случае и `BN_copy()` в простом) за счёт постоянного вытеснения их адресов из кэша. Различие во времени вычисления первой итерации лестницы было усилено примерно до 100 000 и 15 000 циклов для двоичного и простого случая соответственно. Усиление различия во времени сделало возможным детектирование второго старшего бита с высокой вероятностью — около 99% для кривых sect163r1 и P-192. Мы настроили *слот*, то есть время между двумя последовательными зондированиями потоком Flush+Reload, на 5 000 циклов для получения более тонкой гранулярности.

В двоичном случае стратегия детектирования состояла в том, чтобы сначала локализовать в трассах кэш-попадание, соответствующее выполнению первого умножения поля на Z_1 в `gf2m_Madd()` в начале первой итерации лестницы, а затем искать следующее кэш-попадание, соответствующее второму умножению поля, которое отмечает конец первой итерации. Если число слотов между этими двумя было больше 15, это означало различие во времени не менее 75 000 циклов. Первая версия атаки достигла точности 97,3%, которая позднее была улучшена. При запуске атаки на 10 000 вычислениях подписей нам удалось корректно детектировать 2 735 подписей со вторым старшим битом 1 и лишь 27 ложных срабатываний, что составило точность 99,00%. Образцы трасс, иллюстрирующие эту стратегию, можно найти на Рис. 1.

В простом случае стратегия детектирования состояла в поиске первой итерации лестницы путём локализации в трассах кэш-попадания, соответствующего выполнению `ec_GFp_simple_dbl()`, а затем подсчёта числа последовательных кэш-попаданий, соответствующих выполнению `BN_copy()`. Если следующие два слота имели кэш-попадания для последней функции, это означало, что копирование заняло около 3 слотов, или 15 000 циклов. При запуске атаки на 10 000 вычислениях подписей нам удалось корректно детектировать 2 343 подписи со вторым старшим битом 0 и лишь 12 ложных срабатываний, что составило точность 99,53%. Образцы трасс, иллюстрирующие эту стратегию, можно найти на Рис. 2.

Рис. 1. Паттерн в трассах, собранных FR-trace для случая двоичной кривой. Обращения к кэшу считаются попаданиями, когда они ниже стандартного порога в 100 циклов. Кэш-попадания соответствуют выполнению двух первых умножений поля внутри сложения точек, первое — на Z_1 . Когда второй старший бит равен 0 и $Z_1 = 1$ (верхняя трасса), модулярной редукции нет, поэтому два первых умножения поля в сложении точек быстро следуют друг за другом. Когда второй старший бит равен 1 и $Z_1 = x^2$ (нижняя трасса), деградация производительности штрафует модулярную редукцию, и время между двумя умножениями поля становится намного больше.

Рис. 2. Паттерн в трассах, собранных FR-trace для случая простой кривой. Обращения к кэшу снова считаются попаданиями, когда они ниже стандартного порога в 100 циклов. Кэш-попадания соответствуют времени, которое операция `BN_copy()` внутри удвоения точки занимает при деградации производительности. Когда второй старший бит равен 1 (первая трасса), `BN_copy()` не вызывается внутри удвоения точки, но кэш-линия, содержащая вызов функции и следующую операцию поля, подгружается в кэш. Когда второй старший бит равен 0, `BN_copy()` действительно вызывается и занимает больше времени из-за деградации производительности. Паттерн виден между слотами 20 и 30.

3.3. Перевод утечек нонса в экземпляр задачи о скрытом числе

В OpenSSL нонс $k \in \{1, \dots, q - 1\}$ переписывается либо как $\hat{k} = k + q$, либо как $\hat{k} = k + 2q$ перед передачей в алгоритм скалярного умножения, так что результирующий $\hat{k}$ имеет фиксированную битовую длину. Это контрмера против удалённых тайминговых атак Брамли и Тувери [17]. Для кривых с порядком группы чуть ниже степени двойки (обозначаемым $q = 2^l - \delta$) выполняется $\hat{k} = k + q$ за исключением пренебрежимо малой вероятности. Наша атака LadderLeak детектирует второй старший бит $\hat{k}$, и мы утверждаем, что он совпадает с первым старшим битом k с подавляющей вероятностью. Обозначим ℓ -й бит k (соотв. $\hat{k}$) через k_ℓ (соотв. $\hat{k}_\ell$). Тогда $\Pr[k_\ell \neq \hat{k}_\ell] < \Pr[k_\ell = 0 \wedge k < \delta] + \Pr[k_\ell = 1 \wedge k < \delta + 2^{l-\ell-1}]$, поскольку ℓ -й бит q равен 1 и k_ℓ переворачивается только при отсутствии переноса из младших битов в сложении $k + q$. Легко видеть, что правая часть указанного неравенства пренебрежимо мала, если δ пренебрежимо меньше q . Поэтому, объединяя с обычным преобразованием из раздела 2.3, мы получили экземпляры HNP с частотой ошибок не более $\varepsilon = 0,01$ для P-192 и $\varepsilon = 0,027$ для sect163r1.

4. Улучшенный анализ атаки Бляйхенбахера

4.1. Единые компромиссы «время-память-данные»

Подход к HNP на основе FFT обычно требует значительного числа входных подписей по сравнению с атаками на основе решёток. Метод сортировки-и-разности, применённый Араньей и соавт. [6], например, требовал 2^{33} входных подписей для взлома 160-битной ECDSA при 1-битном смещении. Мы могли бы, конечно, применить тот же подход для эксплуатации утечки sect163r1 из предыдущего раздела, но собрать более 8 миллионов подписей через кэш-атаки на практике не так уж легко. Такахаси, Тибуши и Абе [64] применили гораздо более экономичный по памяти подход, используя решатель ранцевой задачи Хоугрейв-Грэма-Жу (HGJ) [35]. Они также приводят «нижние границы» на требуемое число входных образцов для атаки при заданных параметрах подписи и смещениях битов. Однако их формула нижней границы неявно опирается на два искусственных допущения: (1) число входных и выходных образцов, пространственная сложность и размер таблицы FFT все равны (то есть $M = M' = L_{\text{FFT}}$ в Алгоритме 3), и (2) число коллидирующих битов, которые должен найти алгоритм HGJ, фиксировано на некоторой константе. Такие допущения помогают стабилизировать временную и пространственную сложности на протяжении всей атаки, но в то же время, по-видимому, приносят в жертву истинный потенциал применения алгоритма HGJ. В [27] Фук и соавт. кратко упомянули, что алгоритм для GBP помогает снизить число подписей, требуемых в атаках типа Бляйхенбахера, за счёт первоначального усиления числа образцов, хотя их анализ не был ни подробным, ни общим. По сути, HGJ-подобный алгоритм, реализованный в

[64], можно рассматривать как экземпляр обобщённого алгоритма о днях рождения, проанализированного Вагнером [69] и Динуром [25]. Последний, в частности, детально анализирует компромиссы «время-память», которые мы хотели бы применить и расширить, введя третий параметр — сложность по данным. Наша формулировка ниже мотивирована практической ситуацией, в которой злоумышленник может захотеть обменять «онлайн»-затраты на детектирование по стороннему каналу (то есть сложность по данным) на «офлайн»-ресурсы, требуемые атакой Бляйхенбахера (то есть временную и пространственную сложности). Теперь мы намерены рассмотреть следующий вопрос.

При заданной информации о старших битах в HNP и бюджете злоумышленника на вычислительные ресурсы, каков оптимальный баланс между временем, памятью и сложностью по входным данным?

4.1.1. Компромиссы для параметризованного алгоритма суммы 4 списков

Начнём с представления нашего мягкого обобщения формулы компромисса Динура (той, что для алгоритма, обозначенного $A_{4,1}$). Главное отличие в том, что мы сделали число выходных образцов M' произвольным.

Теорема 4.1. Для Алгоритма 4 выполняется следующий компромисс:

$$2^{2^4 M' N} = TM^{2^4} \quad \text{или, иначе говоря,} \quad m' = 3a + v - n$$

где каждый параметр определён следующим образом: $N = 2^n$, где n — число старших битов, подлежащих обнулению; $M = 2^m = 4 \times 2^a$ — число входных образцов, где 2^a — длина каждого подсписка; $M' = 2^{m'} \leq 2^{2a}$ — число выходных образцов, таких что старшие n битов равны 0; $v \in [0, a]$ — параметр, определяющий, сколько итераций поиска коллизий выполняется; $T = 2^t = 2^{a+v}$ — временная сложность.

Доказательство. Для каждого частичного целевого значения $c \in [0, 2^v]$ Шаг 1.a занимает $\tilde{O}(2^a)$ времени и $O(2^m)$ памяти, чтобы найти 2^a пар, дающих в сумме c в старших a битах, например, применяя алгоритм типа сортировки-слияния из [64]. На Шаге 1.b, поскольку две пары x'_1 и x'_2 гарантированно совпадают в старших a битах, вероятность того, что коллизия происходит в старших n битах, равна $1/2^{n-a}$. Следовательно, мы получаем приблизительно $2^{2a}/2^{n-a} = 2^{3a-n}$ линейных комбинаций.⁵ Итерируя эти шаги 2^v раз, мы получаем в общей сложности $M' = 2^{m'} = 2^{3a+v-n}$ образцов за $\tilde{O}(2^{a+v})$ времени и $O(2^m)$ памяти. Поскольку алгоритм проходит не более $2^a \times 2^v$ линейных комбинаций из четырёх, следует, что $2^{m'} \leq 2^{2a}$. ■

Приведённый выше компромисс даёт больше гибкости для усиления числа образцов; как следует из формулы, можно усилить число входных образцов до 2^{2a} за счёт уменьшения обнуляемых битов, или за счёт увеличения временной или пространственной сложности. Это особенно важно в каркасе Бляйхенбахера, поскольку позволяет тщательно координировать число выходных образцов так, чтобы уровень шума был достаточно ниже пика.

⁵ Отметим, что здесь можно получить несколько больше решений благодаря переносам сложений и вычитаний, когда задача инстанцирована над целыми числами (но не над F_2) [64, Теорема 1]. Соответственно приведённый компромисс следует скорректировать на малый постоянный член в правой части, хотя это, конечно, не имеет значения асимптотически.

4.1.2. Интеграция с Бляйхенбахером и линейным программированием

Теперь мы интегрируем приведённую выше базовую формулу компромисса с двумя критическими ограничениями для работы атаки Бляйхенбахера, а именно *малостью* и *разреженностью* выходных линейных комбинаций. В атаке Бляйхенбахера злоумышленник может повторять алгоритм суммы 4 списков r раундов, чтобы найти малые линейные комбинации из 4^r целых чисел ниже некоторого бюджетного параметра для таблицы FFT, $L_FFT = 2^{\{l_FFT\}}$, так чтобы вычисление FFT стало осуществимым. Поэтому мы переписываем формулу компромисса для каждого раунда $i = 0, \dots, r - 1$ как

$$m'_i = 3a_i + v_i - n_i$$

где мы определяем n_i , m_i , m'_i , a_i , v_i и t_i как в Теореме 4.1. Алгоритм 5 описывает итеративный алгоритм суммы 4 списков HGJ, который вызывает Алгоритм 4 в качестве подпрограммы. Отметим, что теперь $2^{m'_i}$ выходов i -го раунда используются как входы $(i + 1)$ -го раунда, поэтому $m_{i+1} = m'_i$. Более того, мы могли бы также включить простую технику фильтрации, обменивающую начальный размер задачи на сложность по данным: имея $2^{m_{in}}$ равномерно случайных l -битных образцов, можно оставить лишь $2^{m_0} = 2^{m_{in}-f}$ образцов ниже $(l - f)$ -бита для любого $f \geq 0$.

С учётом этих обозначений условие малости из Алгоритма 3 выражается как $\log h'_j \leq l - f - \sum_{i=0}^{r-1} n_i \leq l_FFT$, поскольку после r итераций старшие $\sum n_i$ битов $(l - f)$ -битных входных образцов обнуляются. С другой стороны, напомним, что высота пика затухает экспоненциально по L_1 -норме векторов коэффициентов (см. раздел 2.5), поэтому в итоге нужно найти много разреженных линейных комбинаций, чтобы удовлетворить второе условие $|B_q(K)|^{4^r} \gg 1/M'$, где $M' = 2^{m_r}$ — число выходов после r раундов. Введя новую слабую переменную (slack variable) $\alpha \geq 1$ и взяв логарифм, неравенство можно преобразовать в эквивалентное уравнение $m_r = 2(\log \alpha - 4^r \log(|B_q(K)|))$.

Алгоритм 5. Итеративный алгоритм суммы 4 списков HGJ

```
Вход:
L — список из  $M = 4 \times 2^a$  равномерно случайных  $l$ -битных образцов.
{n_i} (i=0..r-1) — число обнуляемых старших битов на раунд.
{v_i} (i=0..r-1) — параметр,  $v_i \in [0, a_i]$ .
Выход: L' — список  $(l - \sum n_i)$ -битных образцов длины  $2^{m_r}$ .
1. Пусть  $a_0 = a$ .
2. Для каждого  $i = 0, \dots, r - 1$ :
  а. Разделить L на 4 непересекающихся списка L1,...,L4 длины  $2^{a_i}$  и отсортировать их.
  б. Применить Алгоритм 4 к {L_i} с параметрами  $n_i$  и  $v_i$ .
  Получить единый список L' ожидаемой длины  $2^{m_{i+1}} = 2^{3a_i+v_i-n_i}$ .
  Положить L := L'.
3. Вывести L'.
```


Здесь мы отмечаем, что слабую переменную α следует определять в зависимости от возможного наибольшего значения шума, которое должно быть несколько больше среднего $1/\sqrt{M}$. Это можно оценить, проверив распределение $\{h'_j\}$ после поиска коллизий в Алгоритме 3: пусть H' и Z' — случайные величины, соответствующие $h'_j = \sum_j \omega_{\{i,j\}} h_i$ и $z'_j = \sum_j \omega_{\{i,j\}} z_i$, и, следовательно, пусть $K' = Z' + \text{sk} \cdot H'$. Поскольку атака Бляйхенбахера должна детектировать пик в точке-кандидате в пределах расстояния $q/(2L_FFT)$ от истинного секрета sk , все модулярные смещения неверного предположения $K'_x = Z' + (\text{sk} \pm x)H' \bmod q$ для $x \in [q/(2L_FFT), q/2]$ являются шумом. Таким образом, наибольшее значение шума есть $\max_x |B_q(K'_x)| = \max_x (|B_q(K')| \cdot |B_q(x \cdot H')|)$ (в силу Леммы 1 из [24]), которое зависит от распределения, которому следует H' . Для каждого конкретного алгоритма поиска коллизий можно экспериментально найти максимальное значение $|B_q(K'_x)|$ и потому выбрать подходящее α , чтобы гарантировать, что пик смещения больше этого значения. Например, для двух раундов итеративного HGJ мы наблюдали $\max_x |B_q(K'_x)| \approx 5/\sqrt{M}$. В Приложении С мы обсуждаем оценку уровня шума более формально.

Объединяя всё вместе, мы получаем единые компромиссы в форме задачи линейного программирования, резюмированной в Таблице 2. Например, чтобы оптимизировать сложность по данным, цель линейного программирования — минимизировать (логарифм) числа входов m_in , при том что задача получает бюджетные параметры t_max , m_max , ℓ_FFT , слабый параметр α и оценённое смещение $B_q(K)$ в качестве фиксированных констант. Мы можем далее итеративно решать линейное программирование по выборам r , чтобы найти оптимальное число раундов, ведущее к наилучшему результату. Для малого числа смещений битов, вроде смещений менее 4 бит, r не превышает 5, поэтому мы можем эффективно найти оптимальные параметры. На Рис. 3 и 4 мы представляем оптимальные сложности по времени и данным для атаки на 1-битно-смещённую HNP при разных размерах таблицы FFT и максимальных границах памяти. Эти результаты получены решением задач линейного программирования с помощью нашего скрипта на SageMath [65], доступного в нашем репозитории на GitHub. Больше графиков компромиссов для случаев утечки нескольких битов нонса также приведено в Приложении D.

Таблица 2. Задачи линейного программирования на основе итеративного алгоритма суммы 4 списков HGJ (Алгоритм 5). Каждый столбец соответствует целевой функции и ограничениям задач линейного программирования для оптимизации сложности по времени, памяти и данным соответственно. Обрамлённые уравнения — общие ограничения для всех задач.

	Время	Память	Данные
минимизировать	$t_o = \dots = t_{\{r-1\}}$	$m_o = \dots = m_{\{r-1\}}$	m_in
при условии	—	$t_i \leq t_max$	$t_i \leq t_max$
при условии	$m_i \leq m_max$	—	$m_i \leq m_max$

Общие ограничения (для всех задач): $-m_{\{i+1\}} = 3a_i + v_i - n_i, i \in [0, r-1] - t_i = a_i + v_i, i \in [0, r-1] - v_i \leq a_i, i \in [0, r-1] - m_i = a_i + 2, i \in [0, r-1] - m_{\{i+1\}} \leq 2a_i, i \in [0, r-1] - m_in = m_o + f - \ell \leq \ell_FFT + f + \sum_{i=0}^{r-1} n_i - m_r = 2(\log \alpha - 4^r \log(|B_q(K)|))$

Одна оговорка состоит в том, что простое итерирование Алгоритма 4 r раундов не обязательно гарантирует 4^r слагаемых в результирующем списке; один и тот же элемент исходного списка может использоваться более одного раза в единственной линейной комбинации из 4^r , когда выходной список i -го раунда используется как вход $(i+1)$ -го раунда. Это нарушило бы ограничения на коэффициенты фазы поиска коллизий, требуемые в Алгоритме 3, поскольку в силу [24, Лемма 1.d] если K следует равномерному распределению над $[0, q/2^l)$, то пик смещения сокращается, то есть $|B_q(2^l K)| = 0$. Чтобы обойти эту проблему, можно вместо этого использовать «многослойный» алгоритм HGJ Динура [25], обобщённый вариант которого мы представляем в Алгоритме 7 вместе с его задачами линейного программирования компромиссов в Таблице 4. При этом входной список сначала делится на 4^r подсписков, и алгоритм гарантирует нахождение линейных комбинаций, составленных из единственного элемента на каждый подсписок, — при этом мы наблюдаем, что конкретные сложности для атаки на наши цели в OpenSSL хуже, чем у итеративного HGJ. На практике несколько итераций алгоритма HGJ выдают пренебрежимо малую долю таких нежелательных линейных комбинаций, и потому фактический пик смещения лишь незначительно ниже оценённого $|B_q(K)|^{\wedge \{4^r\}}$. Эта эвристика также неявно использовалась в [64], и мы решили применять Алгоритм 5 ради лучшей производительности в экспериментах по атаке.

Наконец, отметим, что наш подход универсален и позволяет в принципе интегрировать любые алгоритмы суммы K целочисленных списков для установления аналогичной формулы компромисса «время-память-данные». В Приложении Е мы представляем больше задач линейного программирования, выведенных из других алгоритмов суммы K списков, таких как сумма 16 списков Беккера, Корона и Жу (BCJ) [7] и её многослойный вариант Динура [25]. Для конкретных экземпляров HNP, связанных с нашей атакой на OpenSSL, эти алгоритмы суммы K списков дают несколько худшие сложности, чем итеративный HGJ. Мы оставляем на будущее обнаружение диапазонов параметров, где эти альтернативы работают лучше, а также адаптацию дальнейших алгоритмов суммы списков.

4.2. Функция смещения при наличии ложного детектирования

Все предыдущие работы по атакам на основе FFT [6, 24, 64] рассматривали лишь идеализированную ситуацию, в которой входные образцы HNP не имеют ошибок в информации о старших битах (соответствующей $\varepsilon = 0$ в разделе 2.3). Однако, как отмечено в разделе 3, на практике это обычно не так, поскольку детектирование по стороннему каналу не является 100-процентно точным. Это побуждает нас рассмотреть поведение функции смещения на неравномерно смещённых образцах. Ниже мы показываем, как конкретно вычислять смещения при наличии ε ошибок во входе. Например, наша кэш-тайминговая атака даёт образцы HNP с $\varepsilon = 0,01$ для P-192 (соотв. $\varepsilon = 0,027$ для sect163r1), и настоящая лемма даёт $|B_q(K)| = (1 - 2\varepsilon)|B_q(K_o)| \approx 0,98 \times 0,637$ (соотв. $|B_q(K)| = (1 - 2\varepsilon)|B_q(K_i)| \approx 0,946 \times 0,637$). Отметим, что крайний случай, когда $\varepsilon = 1/2$, просто означает, что образцы вообще не смещены, и потому $|B_q(K)|$ вырождается в 0. Это также согласуется с интуицией: когда злоумышленник не получает никакой информации по стороннему каналу о нонсах, должно быть информационно-теоретически невозможно решить HNP (за исключением наличия некоторой вспомогательной информации вроде знания открытого ключа, соответствующего секрету).

Лемма 4.2. Для $b \in \{0, 1\}$, любого $\varepsilon \in [0, 1/2]$ и чётного целого $q > 0$ выполняется следующее. Пусть K — случайная величина, следующая взвешенному равномерному распределению над $\mathbb{Z}_q$ ниже:

$$\Pr[K = k_i] = (1-b)\cdot\frac{1-\epsilon}{q/2} + b\cdot\frac{\epsilon}{q/2} \quad \text{если } 0 \leq k_i < q/2$$

$$\Pr[K = k_i] = b\cdot\frac{1-\epsilon}{q/2} + (1-b)\cdot\frac{\epsilon}{q/2} \quad \text{если } q/2 \leq k_i < q$$

Тогда модулярное смещение K равно

$$B_q(K) = (1 - 2\epsilon)B_q(K_b)$$

где K_b следует равномерному распределению над $[0 + bq/2, q/2 + bq/2)$.

Доказательство. Докажем случай $b = 0$. Другой случай выполняется по симметрии. По Определению 2.2 смещение для K переписывается благодаря закону бессознательного статистика (LOTUS) в цепочку преобразований, приводящую к

$$B_q(K) = \frac{1-2\epsilon}{q/2}\sum_{k_i \in [0, q/2)} e^{\{2\pi k_i/q\}} = (1 - 2\epsilon)\mathbb{E}\left[e^{\{2\pi K_0/q\}}\right]$$

где $k'_i := k_i - q/2$ и мы использовали $e^{\{2\pi(k'_i+\pi)/q\}} = -e^{\{2\pi k'_i/q\}}$. ■

Ради краткости мы опускаем почти идентичный результат для нечётного q . Отметим, что если q нечётно, то возникает крошечная аддитивная ошибка порядка $1/q$. На практике такая ошибка пренебрежима, поскольку q всегда значительно велико для реальных экземпляров HNP, и мы экспериментально подтвердили, что пик смещения для нечётного q ведёт себя так, как если бы q было чётным.

4.3. Конкретные параметры для атаки на OpenSSL

sect163r1. Чтобы продемонстрировать мощь нашей формулы компромисса, опишем, как конкретно выбрать оптимальные параметры для эксплуатации подверженных ошибкам 1-битных утечек из OpenSSL ECDSA. Для sect163r1 злоумышленник смог бы получить $\ell = 162$ -битные образцы HNP с частотой ошибок не более $\epsilon = 0,027$ благодаря нашему кэш-тайминговому анализу сторонних каналов. По Лемме 4.2 модулярное смещение оценивается как $|B_q(K)| \approx 0,602$. Предположим, что вычислительный бюджет злоумышленника: $t_{max} = 44$, $m_{max} = 29$, $\ell_{FFT} = 34$, и пусть слабая переменная $\alpha = 8$. В следующем разделе мы покажем, что такие вычислительные ресурсы на практике относительно скромны. Если цель злоумышленника — минимизировать число входных образцов m_{in} , то, решая линейное программирование для оптимизации сложности по данным, мы получаем решение $m_{in} = 24$. Наш скрипт-решатель выдаёт все промежуточные параметры атаки, предлагая следующую стратегию, которая усиливает число образцов на 2^5 в первом раунде:

- Первый раунд генерирует $2^{\{m_1\}} = 2^{29}$ образцов с обнулением старших $n_0 = 59$ битов через Алгоритм 4 за время $2^{\{t_0\}} = 2^{\{a_0+v_0\}} = 2^{\{22+22\}} = 2^{44}$, при $2^{\{m_0\}} = 2^{\{m_{in}\}} = 2^{24}$ входных образцах.
- Второй раунд генерирует $2^{\{m_2\}} = 2^{29}$ образцов с обнулением старших $n_1 = 69$ битов через Алгоритм 4 за время $2^{\{t_1\}} = 2^{\{a_1+v_1\}} = 2^{\{27+17\}} = 2^{44}$, при $2^{\{m_1\}} = 2^{29}$ входных образцах.
- После $r = 2$ раундов фазы поиска коллизий фаза вычисления смещения выполняет FFT таблицы размера $2^{\{\ell_{FFT}\}} = 2^{\{\ell-n_0-n_1\}} = 2^{34}$, ожидая обнаружить пик высоты $|B_q(K)|^{\{4^2\}} = \alpha/\sqrt{2^{\{m_2\}}} \approx 0,0003$ и затем восстановить старшие 34 бита sk.

Заметим, что требуемое число входных подписей теперь значительно ниже, чем было бы выведено из предыдущих опубликованных работ. Например, реализация Араньи и соавт. [6] потребовала бы в нашей постановке более 2^{33} входных образцов. Формула нижней границы из [64] при том же слабом параметре давала бы 2^{29} подписей на входе для 2 раундов алгоритма HGJ. Удивительно, но лучший предыдущий рекорд атаки восходит к 2005 году, когда Бляйхенбахер неформально заявил о взломе 160-битной DSA всего при 2^{24} подписях [14], хотя детали никогда не были объяснены до сегодняшнего дня. Поскольку заявленное число образцов совпадает с нашим результатом, наш выбор параметров выше, возможно, объясняет, что он проделал тогда. Более того, все эти работы рассматривали лишь постановку, в которой образцы HNP приходят без каких-либо ошибок в информации о старших битах. В таком идеальном случае наша формула компромисса фактически позволяет осуществить атаку всего при 2^{23} образцах с почти той же сложностью по времени и памяти. Приложение В описывает, как этого можно достичь, в деталях.

NIST P-192. Атака на $\ell = 192$ -битную HNP была бы гораздо более затратной по временной сложности, и злоумышленник, вероятно, захотел бы минимизировать время. Поэтому мы теперь представляем решение задачи линейного программирования для оптимальной временной сложности. Отметим, что наша кэш-атака имела частоту ошибок не более $\epsilon = 0,01$, поэтому оценённый пик смещения несколько отчётливее, чем для sect163r1. Предположим, что злоумышленнику дано $2^{\{m_{in}\}} = 2^{35}$ образцов на входе, а его вычислительный бюджет: $m_{max} = 29$, $\ell_{FFT} = 37$, и теперь положим слабую переменную $\alpha = 16$, чтобы наблюдать более выраженный пик, чем прежде:

- В качестве фазы предобработки отфильтровать $f = 6$ битов, чтобы собрать $2^{\{m_0\}} = 2^{\{m_{in}-f\}} = 2^{29}$ образцов, таких что $h < 2^{\{\ell-f\}} = 2^{186}$.
- Первый раунд генерирует $2^{\{m_1\}} = 2^{29}$ образцов с обнулением старших $n_0 = 75$ битов через Алгоритм 4 за время $2^{\{t_0\}} = 2^{\{a_0+v_0\}} = 2^{\{27+23\}} = 2^{50}$, при $2^{\{m_0\}} = 2^{29}$ входных образцах.
- Второй раунд генерирует $2^{\{m_2\}} = 2^{30}$ образцов с обнулением старших $n_1 = 74$ битов через Алгоритм 4 за время $2^{\{t_1\}} = 2^{\{a_1+v_1\}} = 2^{\{27+23\}} = 2^{50}$, при $2^{\{m_1\}} = 2^{29}$ входных образцах.
- После $r = 2$ раундов фазы поиска коллизий фаза вычисления смещения выполняет FFT таблицы размера $2^{\{\ell_{FFT}\}} = 2^{\{\ell-f-n_0-n_1\}} = 2^{37}$, ожидая обнаружить пик высоты $|B_q(K)|^{\{4^2\}} = \alpha/\sqrt{2^{\{m_2\}}} \approx 0,0005$ и затем восстановить старшие 37 бит sk.

Такая оптимизированная временная сложность позволила нам практически решить ранее не взломанную 192-битную HNP даже при ошибочной 1-битной утечке. Более того, если предположить безошибочный вход, то для решения экземпляра требуется всего 2^{29} образцов при почти той же вычислительной стоимости, как описано в Приложении В. Отметим, что формула нижней границы из [64] при том же слабом параметре, битах фильтрации и модулярном смещении давала бы почти то же число подписей на входе. Однако их непараметризованный алгоритм HGJ исчерпывающе просматривает все битовые паттерны в старших a битах и пытается найти там коллизии (что можно рассматривать как частный случай алгоритма $A_{4,2^m}$ в каркасе Динура при фиксации параметра $v = a$). Результирующий алгоритм работал бы за квадратичное время, приводя к гораздо худшей временной сложности

5. Экспериментальные результаты

5.1. Оптимизированные параллельные реализации

Мы реализовали атаку Бляйхенбахера, инстанцированную с Алгоритмом 5 в качестве метода поиска коллизий. Наша параллельная реализация на основе MPI построена на публично доступной кодовой базе Такахаси и соавт. [63, 64], и мы применили к ней различные оптимизации, которые резюмируем ниже.

- Наша реализация принимает гибкие конфигурации параметров атаки, чтобы полностью учесть компромиссы, наблюдаемые в предыдущем разделе, тогда как [64] позволяла лишь исчерпывающе обнулять фиксированное число битов и не поддерживала никакого усиления образцов.
- После предварительной фазы поиска коллизий в старших a битах между двумя списками мы оставляем лишь старшие 64 бита линейных комбинаций из двух, вместо 128 битов, как делали в [64]. Не теряя корректности алгоритма, это позволяет представлять образцы стандартным типом `uint64_t` и вовсе избежать многоразрядной целочисленной арифметики в последующей фазе поиска коллизий. Благодаря этому изменению как использование ОЗУ, так и счётчики циклов были улучшены вдвое.
- Алгоритм суммы 4 списков требует частой сортировки больших массивов равномерно распределённых случайных элементов (то есть сортировки L'_1 и L'_2 на шаге 1.а Алгоритма 4). В такой ситуации поразрядная сортировка (radix sort) обычно работает лучше алгоритмов сортировки сравнением, вроде быстрой сортировки, использованной в [64]. Используя функцию `spreadsor` библиотеки Boost⁶, мы достигли максимального коэффициента ускорения 1,5.
- В [6] и [64] вычисления FFT проводились на одноузловой машине. Для достижения масштабируемости мы используем интерфейсы FFT с распределённой памятью из FFTW [28].

⁶ https://www.boost.org/doc/libs/1_72_0/libs/sort/doc/html/index.html

5.2. Эксперименты по атаке

NIST P-192. Мы задействовали AWS EC2 для атаки на два экземпляра HNP — без ошибок и с ошибкой $\varepsilon = 0,001$. Конкретные параметры атаки описаны в Приложении В и разделе 4.3 соответственно. Чтобы симулировать подписи ECDSA с утечкой по стороннему каналу, мы сначала получили 2²⁹ и 2³⁵ подписей с (ошибочной) информацией о старших битах нонсов, используя наш собственный интерфейс командной строки на основе модифицированного OpenSSL 1.0.2u (таким образом, что информация о старших битах k раскрывается согласно тому же эмпирическому профилю, построенному в разделе 3), а затем предобработали их для инициализации образцов HNP как в разделе 2.3. Вся генерация подписей заняла 114 CPU-часов и 7 282 CPU-часа в общей сложности для каждого случая соответственно, причём последнее вычисление было распараллелено. Экспериментальные результаты для первой итерации атаки Бляйхенбахера резюмированы в Таблице 3. Для обоих экспериментов мы использовали 24 инстанса `r5.24xlarge` on-demand (с 96 vCPU каждый) для выполнения фазы поиска коллизий. Поскольку текущий крупнейший оптимизированный по памяти инстанс в EC2 — `x1e.32xlarge` (с 4 ТБ ОЗУ)⁷, мы соответственно установили бюджет размера таблицы FFT $L_{\text{FFT}} = 2^{38}$, используя два таких инстанса. Чтобы протестировать как распараллеленный, так и нераспараллеленный FFT, мы запустили 2 узла с распределённой памятью со 128 потоками с общей памятью для первого эксперимента и лишь один поток для второго. В результате нам удалось восстановить ожидаемое число старших битов ключа. Обнаруженные размеры пиков совпали с оценкой $|B_q(K)|^{16}$ с малой относительной ошибкой порядка 2⁻⁵, и наибольшие уровни шума составили около 5-кратного оценённого среднего (то есть $5/\sqrt{2^{m_2}}$) в обоих экспериментах.

⁷ <https://aws.amazon.com/ec2/pricing/on-demand/>

После нахождения старших ℓ' старших битов sk восстановление оставшихся битов достаточно прямолинейно в каркасе Бляйхенбахера: можно просто «повторно внедрить» известную часть секрета в образцы HNP как $k = z + h \cdot sk = (z + h \cdot sk_{\text{hi}} \cdot 2^{\{\ell-\ell'\}}) + h \cdot sk_{\text{lo}}$, где $sk = sk_{\text{hi}} \cdot 2^{\{\ell-\ell'\}} + sk_{\text{lo}}$. Так можно получить новый набор образцов HNP и применить Алгоритм 3 итеративно для восстановления старших битов sk_{lo} . Эти итерации гораздо эффективнее восстановления старших битов, поскольку теперь поиск коллизий должен лишь найти линейные комбинации меньше $2^{\{\ell_{\text{FFT}}+\ell'\}}$ (см. [24] для подробностей). Следуя соглашению из предыдущих работ, мы взяли небольшой запас безопасности: предполагая, что для каждой итерации корректно восстанавливаются лишь $\ell_{\text{FFT}} - 4$ бита, мы установили пространство поиска для неизвестного sk_{lo} на несколько больший интервал $[0, 2^{\{\ell'+4\}}]$. В нашем эксперименте мы запустили Алгоритм 3 в общей сложности 5 раз, пока не были восстановлены старшие 170 бит sk , а затем провели исчерпывающий перебор для нахождения оставшихся 22 битов. Все итерации, кроме первой, были завершены примерно за 6 часов с использованием Intel Xeon E5-2670 CPU ×2 (16 ядер в общей сложности) со 128 ГБ ОЗУ.

Таблица 3. Сводка экспериментальных результатов. Столбцы «Поток» имеют формат «число потоков с общей памятью × число узлов с распределённой памятью». «Восстановленные старшие биты» вычислялись относительно относительной ошибки от истинного секрета sk , то есть $|\ell - \log |sk - w||$, где w — оценённый секретный ключ, а ℓ — битовая длина размера группы. Отметим, что большой объём потребления памяти обусловлен накладными расходами распараллеливания, и фактически использование ОЗУ на поток было ниже 6 ГБ в фазе поиска коллизий и 32 ГБ в FFT соответственно.

Цель	Ресурс	Стоимость	Частота ошибок	Вход	Выход	Поток (колл.)	Время (колл.)	ОЗУ (колл.)	L_FFT	Поток (FFT)	Время (FFT)	ОЗУ (FFT)	Высота пика	Макс. шум	Восст. ст. биты
NIST P-192	AWS EC2	\$16 429	0	2 ²⁹	2 ²⁷	96 × 24	113 ч	492 ГБ	2 ³⁸	128 × 2	0,5 ч	4 ТБ	7,28·10 ⁻⁴	4,48·10 ⁻⁴	39
NIST P-192	AWS EC2	\$7 870	0,010	2 ³⁵	2 ³⁰	96 × 24	52 ч	492 ГБ	2 ³⁷	1	12 ч	4 ТБ	5,04·10 ⁻⁴	1,55·10 ⁻⁴	39
sect163r1	Кластер	–	0	2 ²³	2 ²⁷	16 × 16	7 ч	80 ГБ	2 ³⁵	8 × 8	1 ч	128 ГБ	4,92·10 ⁻⁴	4,29·10 ⁻⁴	36
sect163r1	Рабочая станция	–	0,027	2 ²⁴	2 ²⁹	48	42 ч	250 ГБ	2 ³⁴	16	1 ч	512 ГБ	2,82·10 ⁻⁴	2,21·10 ⁻⁴	35

sect163r1. Мы задействовали узлы параллельного кластера (Intel Xeon E5-2670) и рабочую станцию (Intel Xeon E5-2697) для атаки на два экземпляра HNP — без ошибок и с ошибкой $\varepsilon = 0,0027$. Конкретные параметры атаки для первого описаны в Приложении В, а для второго уже были описаны в разделе 4.3. Мы сначала сгенерировали 2²³ и 2²⁴ подписи ECDSA, как в случае P-192, что заняло 1,8 и 3,6 CPU-часа соответственно. Измеренные экспериментальные результаты — в Таблице 3. Восстановление оставшихся битов также было проведено и заняло около 2 часов с использованием единственного вычислительного узла. Мы наблюдали, что высота пика после поиска коллизий для sect163r1 без ошибок ниже оценённого $|B_q(K)|^{16} \approx 7,3 \cdot 10^{-4}$. Мы предполагаем, что это могло быть вызвано вторым раундом HGJ, применяемым к распределениям, которые уже не равномерны, хотя формальный анализ оставлен на будущее. Благодаря нашей оптимизированной реализации обе атаки удались при относительно скромных вычислительных затратах по сравнению с предыдущими работами. Поскольку время CPU составляет менее 3 месяцев, а использование памяти на поток было 32 ГБ в обоих случаях, мы можем заключить, что всю атаку можно было бы легко выполнить на современном нодбуке.

Оценки стоимости AWS для атаки на NIST P-224 и P-256. Рис. 4 указывает, что при 2³⁵ подписях P-224 и 2³⁵ памяти можно завершить фазу поиска коллизий за 2^{54,5} времени и затем решить HNP, вычислив FFT размера 2⁴⁵. Поэтому из приведённых эмпирических результатов можно вывести конкретные затраты AWS на взлом P-224 всего при 1-битных утечках нонса; действительно, всё вычисление для такого случая можно было бы завершить, заплатив около \$300 000 Amazon и запустив 256 инстансов xle.32xlarge на 45 дней (даже с учётом накладных расходов распараллеливания), что должно быть практически осуществимо для хорошо финансируемых противников. Взлом P-256 с 1-битной утечкой остаётся сложным; однако, если доступны 2 бита утечки благодаря более мощной атаке по стороннему каналу, то Рис. 6 подразумевает, что восстановление ключа осуществимо при тех же вычислительных/AWS-затратах, при примерно полумиллионе подписей.

6. Программные контрмеры

Основная контрмера для защиты от нашей атаки — обеспечение постоянно-временного поведения в реализации скалярного умножения. Мы обсуждаем три варианта в порядке возрастания сложности реализации: рандомизация Z-координаты, постоянно-временное вычисление группового закона и альтернативные алгоритмы скалярного умножения.

При наличии высококачественного источника энтропии, также необходимого для генерации нонсов ECDSA, наиболее просто реализуемая контрмера — рандомизация Z-координат для гарантии того, что все промежуточные точки находятся в проективных координатах. Это популярная контрмера при реализации лестницы Монтгомери в Curve25519, хотя её точная эффективность против атак по сторонним каналам в этом контексте не вполне ясна [26]. Мы также отмечаем, что дополнительная осторожность необходима при переходе от проективных к аффинным координатам в конце вычисления, чтобы предотвратить связанную атаку [19].

Другая потенциальная контрмера — рефакторинг реализации для удовлетворения гарантий постоянного времени. Первый вариант — реализовать групповой закон с постоянным временем, используя полные формулы из [54], допуская существенное влияние на производительность [61]. Существуют и другие альтернативы для алгоритма скалярного умножения, которые не так сильно наказывают производительность. Например, устойчивая к SPA стратегия скалярного умножения «слева-направо удвоение-и-сложение» Корона [20] вычисляет сложение точки и удвоение точки на каждой итерации, используя условное копирование для выбора корректного результата в конце. Эта стратегия имела бы сопоставимую с лестницей Монтгомери производительность в модели Вейерштрасса, будучи при этом концептуально проще для безопасной реализации. Другими альтернативами были бы реализация лестницы над со-Z арифметикой для устранения явной обработки Z-координат [44] или специальная свободная от исключений лестница, благоприятствующая постоянно-временному выполнению [62].

Наши патчи, представленные в рамках скоординированного раскрытия, реализуют контрмеру рандомизации координат для независимой рандомизации обоих аккумуляторов в качестве меры глубоко эшелонированной защиты, без необходимости учитывать, как реализована лежащая в основе арифметика поля. Мы подтвердили эффективность контрмеры как на двоичных, так и на простых кривых, не сумев осуществить те же кэш-тайминговые атаки против пропатченных реализаций, а позднее — запустив инструмент динамического анализа duedect [56]. Наши патчи, иллюстрирующие контрмеру, доступны в нашем репозитории на GitHub вместе с наборами данных для вычисления подписей, содержащими кэш-тайминговые трассы и интеграцию с duedect.

Следуя недавним тенденциям в исследованиях и практике, мы рекомендуем новым приложениям цифровых подписей принимать более современные схемы, облегчающие постоянно-временную реализацию [11].

Благодарности

Это исследование было поддержано: Concordium Blockchain Research Center, Орхусский университет, Дания; Фондом Карлсберга в рамках исследовательского проекта Semper Ardens CF18-112 (BCM); Европейским исследовательским советом (ERC) в рамках программы исследований и инноваций Horizon 2020 Европейского союза по грантовому соглашению № 803096 (SPEC); Датским независимым исследовательским советом по гранту DFF-6108-00169 (FoCC); премией Australian Research Council Discovery Early Career Researcher Award (проект № DE200101577); подарком от Intel. Мы благодарим команду разработчиков OpenSSL и анонимных рецензентов за их ценные комментарии и предложения. Часть вычислений для этого проекта была выполнена на кластере GenomeDK. Мы хотели бы поблагодарить GenomeDK и Центр анализа генома и персонализированной медицины (CGPM) за предоставление вычислительных ресурсов и поддержки, способствовавших этим результатам исследования.

Приложение А. Постоянно-временная лестница Монтгомери

Приведённый ниже Алгоритм 6 удовлетворяет условиям, перечисленным в разделе 2.2, для реализации лестницы Монтгомери с постоянным временем. Выбор, условные перестановки и групповой закон, естественно, должны быть реализованы с постоянным временем, следуя соответствующей литературе [26, 54].

Алгоритм 6. Постоянно-временная лестница Монтгомери

```
Вход:  $k \in \mathbb{Z}_q$ , точка  $P$  на  $E(F)$ .
Выход: результат скалярного умножения  $R = [k]P$ .
1:  $(R_0, R_1) \leftarrow (P, 2P)$ 
2:  $\hat{k} \leftarrow \text{Select}(k + q, k + 2q)$ 
3: для  $i = \lfloor \lg(q) \rfloor$  до 0:
4:    $\text{Swap}(R_0, R_1)$ , если  $\hat{k}_i = 0$ 
5:    $(R_0, R_1) = ([2]R_0, R_0 \oplus R_1)$ 
6:    $\text{Swap}(R_0, R_1)$ , если  $\hat{k}_i = 0$ 
7: конец цикла
8: вернуть  $R_0$ 
```

Приложение В. Конкретные параметры для атаки на HNP без ошибок

sect163r1. Для $\ell = 162$ мы представляем решение задачи линейного программирования из Таблицы 2 для оптимизированной сложности по данным. Предположим, злоумышленнику дано $2^{\{m_{in}\}} = 2^{23}$ образцов HNP с $\varepsilon = 0$ на входе, а его вычислительный бюджет: $m_{max} = 29$, $\ell_{FFT} = 35$, и пусть слабая переменная $\alpha = 8$.

- Первый раунд генерирует $2^{\{m_1\}} = 2^{29}$ образцов с обнулением старших $n_0 = 55$ битов через Алгоритм 4 за время $2^{\{t_0\}} = 2^{\{a_0+v_0\}} = 2^{\{21+21\}} = 2^{42}$, при $2^{\{m_0\}} = 2^{\{m_{in}\}} = 2^{23}$ входных образцах.
- Второй раунд генерирует $2^{\{m_2\}} = 2^{27}$ образцов с обнулением старших $n_1 = 72$ битов через Алгоритм 4 за время $2^{\{t_1\}} = 2^{\{a_1+v_1\}} = 2^{\{27+18\}} = 2^{45}$, при $2^{\{m_1\}} = 2^{29}$ входных образцах.
- После $r = 2$ раундов фазы поиска коллизий фаза вычисления смещения выполняет FFT таблицы размера $2^{\{\ell_{FFT}\}} = 2^{\{\ell-n_0-n_1\}} = 2^{35}$, ожидая обнаружить пик высоты $|B_q(K)|^{4^2} = \alpha/\sqrt{2^{\{m_2\}}} \approx 0,0007$ и затем восстановить старшие 35 бит sk.

P-192. Для $\ell = 192$ мы представляем решение задачи линейного программирования из Таблицы 2 для оптимизированной временной сложности. Предположим, злоумышленнику дано $2^{\{m_{in}\}} = 2^{29}$ образцов HNP с $\varepsilon = 0$ на входе, а его вычислительный бюджет: $m_{max} = 29$, $\ell_{FFT} = 38$, и пусть слабая переменная $\alpha = 8$.

- Первый раунд генерирует $2^{\{m_1\}} = 2^{29}$ образцов с обнулением старших $n_0 = 76$ битов через Алгоритм 4 за время $2^{\{t_0\}} = 2^{\{a_0+v_0\}} = 2^{\{27+24\}} = 2^{51}$, при $2^{\{m_0\}} = 2^{\{m_{in}\}} = 2^{29}$ входных образцах.
- Второй раунд генерирует $2^{\{m_2\}} = 2^{27}$ образцов с обнулением старших $n_1 = 78$ битов через Алгоритм 4 за время $2^{\{t_1\}} = 2^{\{a_1+v_1\}} = 2^{\{27+24\}} = 2^{51}$, при $2^{\{m_1\}} = 2^{29}$ входных образцах.
- После $r = 2$ раундов фазы поиска коллизий фаза вычисления смещения выполняет FFT таблицы размера $2^{\{\ell_{FFT}\}} = 2^{\{\ell-n_0-n_1\}} = 2^{38}$, ожидая обнаружить пик высоты $|B_q(K)|^{4^2} = \alpha/\sqrt{2^{\{m_2\}}} \approx 0,0007$ и затем восстановить старшие 38 бит sk.

Приложение С. Оценка высоты уровня шума

В этом приложении мы опираемся на стандартные результаты о субгауссовых случайных величинах, введённых, например, в [16]. В частности, мы используем следующий результат. В цитируемой работе он сформулирован над $\mathbb{R}$, но обобщение на $\mathbb{C}$ немедленно.

Лемма С.1 ([16, Лемма 4.4]). Пусть $\vec{v}$ — t -субгауссов случайный вектор в $\mathbb{C}^N$. Тогда:

$$\Pr[\|\vec{v}\|_{\infty} > t] \leq 2N \cdot \exp\left(-\frac{t^2}{2\tau^2}\right).$$

Теперь рассмотрим $H' = \{h'_i\}$ — семейство из M независимых одинаково распределённых случайных величин над $\mathbb{Z}_q$, и предположим, что для некоторого $x \in \mathbb{Z}_q$ выполняется, что модулярное смещение распределения $x \cdot h'_i$ равно нулю. Это, например, выполняется, когда h'_i равномерны над $\{0, \dots, L - 1\}$ для некоторого L , делящего q , и x — ненулевое кратное q/L . Выборочное смещение:

$$B_q(x \cdot H') = \frac{1}{M} \sum_{i=1}^M e^{2i\pi i x \cdot h'_i/q}$$

является линейной комбинацией случайных величин $u_i = e^{2i\pi x \cdot h'_i/q}$, которые независимы, имеют среднее 0 по предположению о модулярном смещении и лежат на единичной окружности, а потому являются 1-субгауссовыми по лемме

Хёфдинга (см. [16, Лемма 2.5(iv)]). Следовательно, $B_q(x \cdot H)$ сам по себе является τ -субгауссовым для $\tau = \sqrt{(1/M^2 + \dots + 1/M^2)} = 1/\sqrt{M}$.

Дано семейство X точек x как выше; предположим, что субгауссовы случайные величины $B_q(x \cdot H)$, $x \in X$, независимы.⁸ Тогда вектор $\vec{v} = (B_q(x \cdot H))_{x \in X}$ снова является $1/\sqrt{M}$ -субгауссовым. В частности, по Лемме С.1 имеем:

$$\Pr[\|\vec{v}\|_{\infty} > t] \leq 2|X| \cdot e^{-Mt^2/2}.$$

Иными словами, за исключением вероятности не более ε , имеем:

$$\max_{x \in X} |B_q(x \cdot H)| \leq \sqrt{\frac{2 \ln(2|X|/\varepsilon)}{M}}.$$

При выполнении поиска пика на основе FFT мы находимся в постановке, где L — число точек FFT, а X — множество из $L - 1$ ненулевых кратных q/L . Аргумент показывает, что мы ожидаем, что уровень шума будет не более чем в $\sqrt{(2 \ln(2L/\varepsilon))}$ раз выше среднего $1/\sqrt{M}$.

В качестве примера, при $L = 2^{35}$ и $\varepsilon = 2^{-10}$ это даёт верхнюю границу чуть ниже 8. Поэтому выбор слабого значения несколько выше 8 для FFT из 2^{35} точек должен обеспечить корректное восстановление пика с высокой вероятностью.

⁸ Можно, но нетривиально, количественно оценить, в какой мере это допущение выполняется: см., например, [59]. В нашей постановке оно представляется разумным эвристическим допущением.

Приложение D. Улучшенные оценки сложности для HNP с бóльшим числом утечек нонса

Для атак на OpenSSL ECDSA мы сосредоточились на анализе случаев (менее чем) однобитных утечек нонса. В этом приложении мы рассматриваем менее ограниченные ситуации, в которых злоумышленнику доступно несколько битов нонсов на подпись. Насколько нам известно, все ранее обнаруженные уязвимости ECDSA в литературе требовали не менее 3 битов нонсов для практического осуществления атак на решётках (за исключением 160-битной DSA с 2-битными утечками, взломанной [41]). Благодаря нашей формуле компромисса мы показываем, что 2 или 3 бита утечек нонса действительно эксплуатируемы при скромной сложности по входным данным, сохраняя при этом реалистичные сложности по времени и памяти (напомним, что наши эксперименты по атаке для P-192 практически решили случай, соответствующий более чем 2^{50} времени и 2^{29} памяти). Например, высокопрофильная кривая NIST P-256 может быть легко взломана при всего нескольких тысячах подписей, утекающих 3 бита нонсов каждая. Мы включаем на Рис. 5–8 компромиссы «время–данные» для различных комбинаций параметров: целевого размера группы, числа смещённых битов нонса, максимальной границы памяти и размера таблицы FFT. Все результаты получены решением задачи линейного программирования из Таблицы 2.

Рисунки 3–8 представляют графики компромиссов «время–данные» для различных размеров кривых (sect163r1, P-192, P-224, P-256), при разных величинах смещения нонса (1, 2, 3 бита), при $m_{\max} = 30$ или 35, слабом параметре $\alpha = 8$ и разном числе раундов r . Каждая кривая на графиках соответствует размеру таблицы FFT $\ell_{\text{FFT}} \in \{35, 40, 45\}$.

Приложение E. Дополнительные алгоритмы суммы списков и формулы компромиссов

Ниже мы представляем альтернативные алгоритмы суммы K списков (Алгоритмы 7, 9 и 10), которые могут быть применены к фазе поиска коллизий вместо итеративного HGJ из Алгоритма 5. Соответствующие задачи линейного программирования, представляющие компромиссы, приведены в Таблицах 4–6. Скрипт-решатель для всех задач также доступен в нашем репозитории на GitHub.

Таблица 4. Задачи линейного программирования на основе многослойного алгоритма суммы $K = 4^r$ списков (Алгоритм 7).

Целевые функции: минимизировать $t_o = \dots = t_{r-1}$ (время) / $m_o = \dots = m_{r-1}$ (память) / $2^{m_{\min}}$ (данные), при аналогичных ограничениях по $t_{\max}$ и $m_{\max}$.

Общие ограничения: $-a_{i+1} = 3a_i + v_i - n_i, i \in [0, r-1] - t_i = a_i + v_i + 2(r-i-1), i \in [0, r-1] - v_i \leq a_i, i \in [0, r-1] - m_i = a_i + 2(r-i), i \in [0, r-1] - a_{i+1} \leq 2a_i, i \in [0, r-1] - m_{\min} = m_o + f - \ell \leq \ell_{\text{FFT}} + f + \sum n_i - m_r = 2(\log \alpha - 4^r \log(|B_q(K)|))$

Таблица 5. Задачи линейного программирования на основе итеративного алгоритма суммы 16 списков BSJ (Алгоритм 9).

Общие ограничения: $-m_{i+1} = 7a_i + w_i - n_i, i \in [0, r-1] - t_i = 2a_i + w_i, i \in [0, r-1] - w_i \leq 9a_i, i \in [0, r-1] - m_i = a_i + 4, i \in [0, r-1] - m_{i+1} \leq 11a_i, i \in [0, r-1] - m_{\min} = m_o + f - \ell \leq \ell_{\text{FFT}} + f + \sum n_i - m_r = 2(\log \alpha - 16^r \log(|B_q(K)|))$

Таблица 6. Задачи линейного программирования на основе многослойного алгоритма суммы 16 списков BSJ (Алгоритм 10).

Общие ограничения: $-a_{i+1} = 7a_i + w_i - n_i, i \in [0, r-1] - t_i = 2a_i + w_i + 4(r-i-1), i \in [0, r-1] - w_i \leq 9a_i, i \in [0, r-1] - m_i = a_i + 4(r-i), i \in [0, r-1] - a_{i+1} \leq 11a_i, i \in [0, r-1] - m_{\min} = m_o + f - \ell \leq \ell_{\text{FFT}} + f + \sum n_i - m_r = 2(\log \alpha - 16^r \log(|B_q(K)|))$

Алгоритм 7. Параметризованный многослойный алгоритм суммы $K = 4^r$ списков на основе Динура [25]

Вход:
 L – список из $M = 4^g \times 2^a$ равномерно случайных ℓ -битных образцов.
 $\{n_i\}, \{v_i\}$ ($i=0..g-1$) – число обнуляемых старших битов и параметр ($v_i \in [0, a_i]$).
Выход: L' – список ($\ell - \sum n_i$)-битных образцов длины $2^{\{m_g\}}$.
1. Разделить L на 4^g непересекающихся списков $L_1, \dots, L_{\{4^g\}}$. Пусть $a_0 = a$.
2. Для каждого $i = 0, \dots, g - 1$:
а. Отсортировать списки $L_1, \dots, L_{\{4^{\{g-i\}}\}}$ и применить Алгоритм 4 $4^{\{g-i-1\}}$ раз к группам по 4 списка с параметрами n_i и v_i .
Получить $4^{\{g-i-1\}}$ списков ожидаемой длины $2^{\{a_{i+1}\}} = 2^{\{3a_i + v_i - n_i\}}$.
б. Положить $L_1 := L'_1, \dots$
3. Вывести L_1 .

Алгоритм 8. Параметризованный алгоритм суммы 16 списков на основе Беккера-Корона-Жу [7]

Вход:
 $\{L_i\}$ ($i=1..16$) – отсортированные списки из 2^a равномерно случайных ℓ -битных образцов.
 n – число обнуляемых старших битов; $w \in [0, 9a]$ – параметр.
Выход: L' – список ($\ell - n$)-битных образцов.
1. Для каждого из 2^w возможных значений значений четырёх 3а-битных слов c_1, c_2, c_3, c_4 , удовлетворяющих $c_1 + c_2 - c_3 - c_4 = 0$:
а. Применить Алгоритм 4 четыре раза к группам списков с параметрами $n = 3a, v = a$ и целевыми значениями c_1, c_2, c_3, c_4 .
Сохранить ожидаемое число $2^{\{4a-3a\}} = 2^a$ выходных сумм в новые списки L'_1, L'_2, L'_3, L'_4 .
б. Применить Алгоритм 4 к $L'_1, \dots, L'_4$ с параметрами $n = n - 3a$ и $v = a$.
Сохранить ожидаемое число $2^{\{4a-(n-3a)\}} = 2^{\{7a-n\}}$ выходных сумм в L' .
2. Вывести L' ожидаемой длины $2^{\{7a+w-n\}}$.

Алгоритм 9. Итеративный алгоритм суммы 16 списков ВСJ

Вход:
 L – список из $M = 16 \times 2^a$ равномерно случайных ℓ -битных образцов.
 $\{n_i\}, \{w_i\}$ ($i=0..g-1$) – число обнуляемых старших битов и параметр ($w_i \in [0, 9a_i]$).
Выход: L' – список ($\ell - \sum n_i$)-битных образцов длины $2^{\{m_g\}}$.
1. Пусть $a_0 = a$.
2. Для каждого $i = 0, \dots, g - 1$:
а. Разделить L на 16 непересекающихся списков длины $2^{\{a_i\}}$ и отсортировать.
б. Применить Алгоритм 8 с параметрами n_i и w_i .
Получить единый список L' длины $2^{\{m_{i+1}\}} = 2^{\{7a_i + w_i - n_i\}}$. Положить $L := L'$.
3. Вывести L' .

Алгоритм 10. Параметризованный многослойный алгоритм суммы $K = 16^r$ списков на основе Динура [25]

Вход:
 L – список из $M = 16^g \times 2^a$ равномерно случайных ℓ -битных образцов.
 $\{n_i\}, \{w_i\}$ ($i=0..g-1$) – число обнуляемых старших битов и параметр ($w_i \in [0, 9a_i]$).
Выход: L' – список ($\ell - \sum n_i$)-битных образцов длины $2^{\{m_g\}}$.
1. Разделить L на 16^g непересекающихся списков $L_1, \dots, L_{\{16^g\}}$. Пусть $a_0 = a$.
2. Для каждого $i = 0, \dots, g - 1$:
а. Отсортировать списки и применить Алгоритм 8 $16^{\{g-i-1\}}$ раз к группам по 16 списков с параметрами n_i и w_i .
Получить $16^{\{g-i-1\}}$ списков длины $2^{\{a_{i+1}\}} = 2^{\{7a_i + w_i - n_i\}}$.
б. Положить $L_1 := L'_1, \dots$
3. Вывести L_1 .

Литература

Полный список литературы (ссылки [1]–[73]) приведён в оригинальной статье. Ключевые источники включают: Боне и Венкатесан (HNP, CRYPTO'96) [15]; Бляйхенбахер (генерация одноразовых ключей, IEEE P1363, 2000; «Эксперименты с DSA», CRYPTO 2005) [13, 14]; Хоугрейв-Грэм и Сمارт (решёточные атаки на схемы подписи) [36]; Аранья и соавт. (GLV/GLS-разложение и атаки на ECDSA при однобитном смещении нонса, ASIACRYPT 2014) [6]; Такахаси, Тибуши и Абе (новые рекорды Бляйхенбахера, TCHES 2018) [64]; Динур (алгоритмический каркас для обобщённой задачи о днях рождения, 2019) [25]; Вагнер (обобщённая задача о днях рождения, CRYPTO 2002) [69]; Хоугрейв-Грэм и Жу (новые обобщённые алгоритмы для трудных ранцев, EUROCRYPT 2010) [35]; Беккер, Корон и Жу (улучшенные обобщённые алгоритмы для трудных ранцев, EUROCRYPT 2011) [7]; Яром и Фолкнер (FLUSH+RELOAD, USENIX Security 2014) [73].